

Graph Algorithms

1

Graph Algorithms

Topics

- *Graph terminology*
- *Graph Representations*
 - *Matrix Representation*
 - *Linked list Representation*
- *Graph Traversal*
 - *Breath First Search (BFS)*
 - *BFS Analysis*
 - *Depth First Search (DFS)*
 - *DFS Analysis*
- *Spanning Trees*
 - *Kruskal's algorithm and analysis*
 - *Prim's algorithm*
- *Single shortest distance*
 - *Dijkstra's algorithm*
- *Topological Sorting*

2

Graphs

Terminology

A **graph** $G = (V, E)$ is a set of vertices V and set of edges E , where $E \subseteq V \times V$.

•An example of a graph with vertices v_1, v_2, v_3, v_4, v_5 is

$$\rightarrow G = (V, E)$$

$$\rightarrow V = \{v_1, v_2, v_3, v_4, v_5\}$$

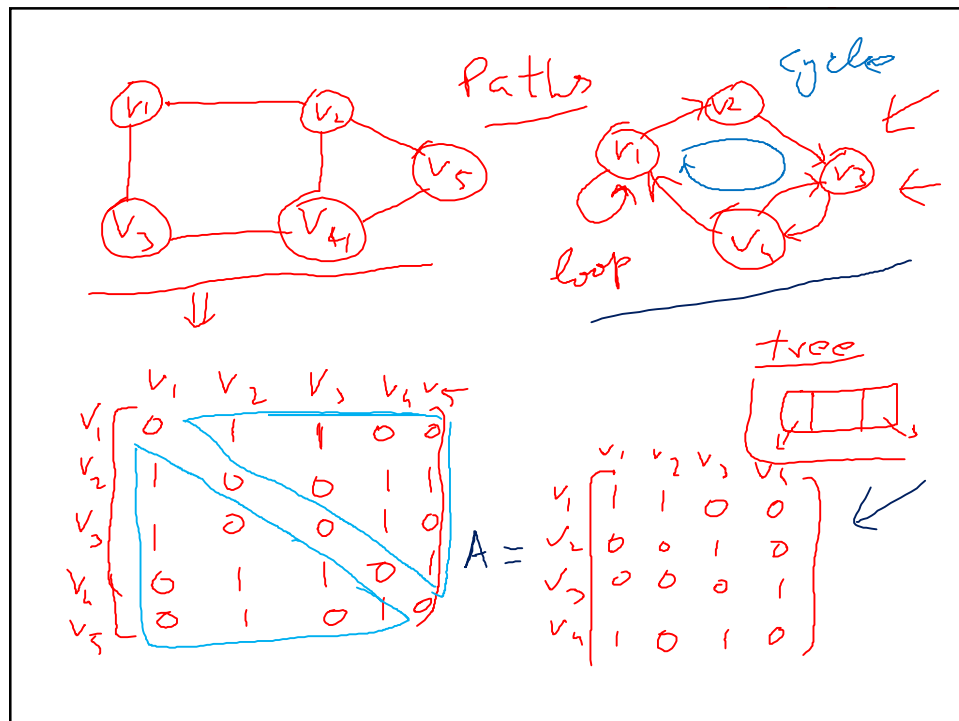
$$E = V \times V$$

$$= \{(v_1, v_1), (v_1, v_2), (v_1, v_3), (v_1, v_4), (v_1, v_5), \\ (v_2, v_1), (v_2, v_2), (v_2, v_3), (v_2, v_4), (v_2, v_5), \\ (v_3, v_1), (v_3, v_2), (v_3, v_3), (v_3, v_4), (v_3, v_5), \\ (v_4, v_1), (v_4, v_2), (v_4, v_3), (v_4, v_4), (v_4, v_5), \\ (v_5, v_1), (v_5, v_2), (v_5, v_3), (v_5, v_4), (v_5, v_5)\}$$

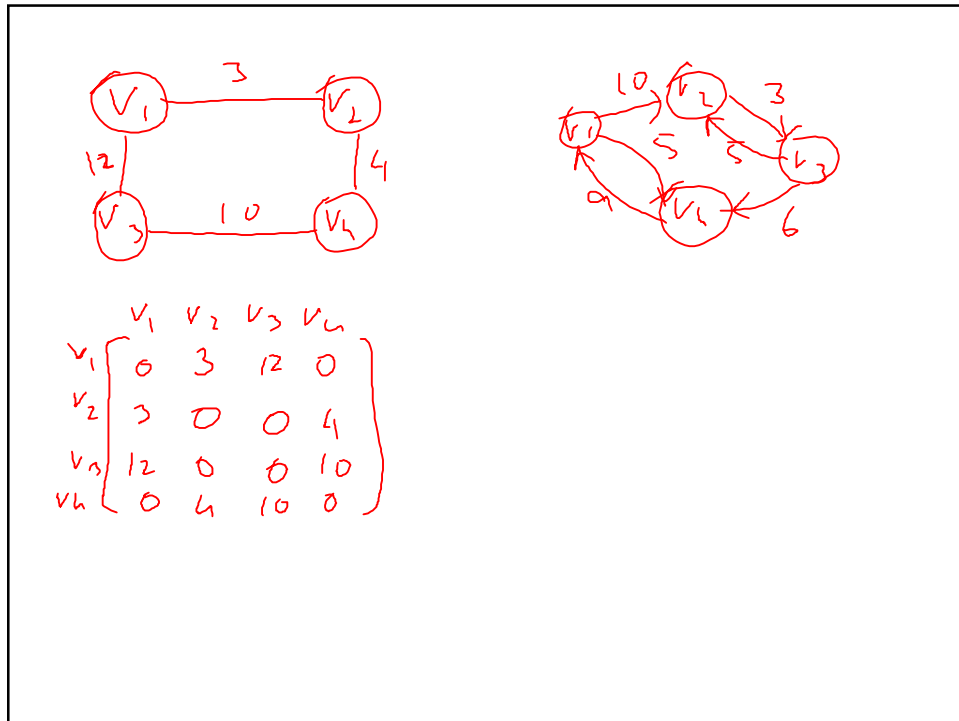


•The number of vertices and edges in the graph is given by the set **cardinalities** of the sets V, G . In the example, $|V| = 5, |E| = 25$

3



4



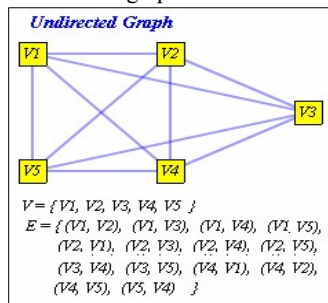
5

Graphs

Undirected

A graph $G=(V,E)$ with vertex set $V=\{v_1, v_2, v_3, \dots, v_n\}$ is called **undirected** if $(v_i, v_j) = (v_j, v_i)$ for $i \neq j$

- An undirected graph is sometimes referred to as **undigraph**.
- In pictorial representation of undirected graph, the edges are not assigned any direction. Figure shows an example of undirected graph.



• A graph which has all the edges is referred to as **complete**. An **undirected complete graph**, with n vertices, has $n(n-1)/2$ edges. Its **space complexity** is $O(n^2)$. Such a graph is called **dense**. By contrast, a graph with space complexity $O(n \lg n)$ is called **sparse**.

zzz

6

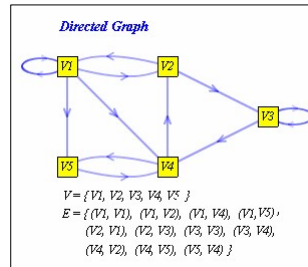
Graphs

Directed

A graph $G=(V,E)$ with vertex set $V=\{v_1, v_2, v_3, \dots, v_n\}$ is called **directed** or **digraph** if $(v_i, v_j) \neq (v_j, v_i)$ for $i \neq j$

In other words edges (v_i, v_j) and (v_j, v_i) , associated with any pair of vertices v_i, v_j , are considered **distinct**. In pictorial representation these are shown with arrows.

•The directed graph is often referred to digraph or network. Figure shows an example of directed graph



•A **complete directed graph** with n vertices has n^2 edges, including edges associated with all individual vertices. If individual edges are excluded, the total number of edges is $n(n-1)$

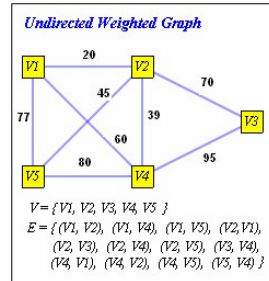
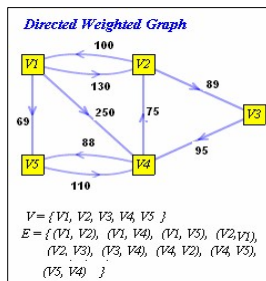
7

Graphs

Weighted

A graph in which **values** $w_1, w_2, w_3, \dots$ are associated with edges is called **weighted** graph. Weights are typically **costs** or **distances** in different applications of graphs.

•A weighted graph can be **directed** or **undirected**. Figures show examples of such graphs.



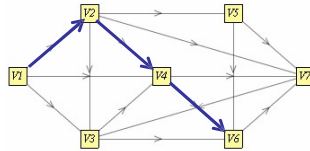
•Weighted graphs have several real life applications, such as air routes, computer networks, road links

8

Graphs

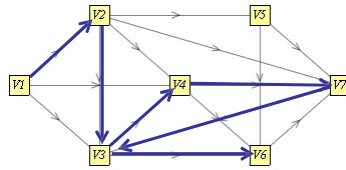
Paths

A **path** is a sequence of vertices which are connected by the consecutive edges.
The number of edges is called the **path length**. For example, in the figure the path $\{v1, v2, v4, v6\}$ has length 3



(a) Path length of 3

- In **simple path** no vertices are repeated. The path $\{v1, v2, v4, v6\}$ is simple. The path $\{v1, v3, v4, v7, v3, v6\}$ is **non-simple**, because it passes through vertex $v3$ twice



(b) Non-simple path

9

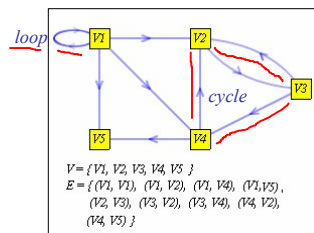
Graphs

Loops and Cycles

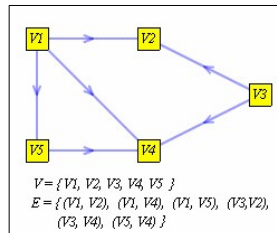
A **loop** is special path that originates and terminates at a single node. In the sample graph a loop is associated with vertex $v1$.

A path that originates and terminates at the same vertex, and links two or vertices, is called **cycle**. In the picture of the cyclic graph the path $v2, v3, v4, v2$ is cycle.

If a graph contains a cycle it is called **cyclic**. By contrast, a graph which contains no cycles is known as **acyclic**.



(a) cyclic graph



acyclic graph

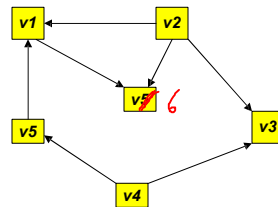
10

Graphs

In-degree and Out-degree

The number of edges that connect a given vertex to other vertices is called **degree of the vertex**. In a directed graph, the number of edges that originate from a vertex is called **out-degree**. Likewise, the number of edges that terminate on vertex is called **in-degree** of the vertex.

▪ In the figure, the **in-degree** of vertex $v1$ is 2 and **out-degree** is 1.



In-degrees and Out-degrees

11

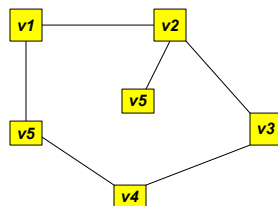
Graphs

Connectedness

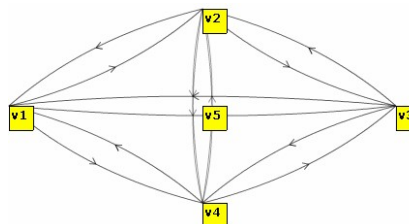
▪ A graph is said to be **connected** if there exists a path between all pairs of vertices. In other words, each vertex is **reachable** from any other vertex in the graph.

▪ Connectedness in an undirected graph is simple because the edges can be traversed in either direction to reach a vertex.

▪ Figures show a **connected undirected graph** and **connected directed graphs**



(a) Connected undirected graph



(b) Connected directed graph

12

Graphs

Matrix Representation

One of the standard ways of representing graph is to use a matrix to denote links between pairs of vertices in a graph. The matrix is known as adjacency matrix.

Let $G=(V,E)$ be a graph, with $V=\{v_1, v_2, \dots, v_n\}$, and $E=\{(v_1, v_2), (v_1, v_3), \dots\}$ where $n=|V|$. The adjacency matrix $A=(a_{ij})$ as

$$a_{ij} = \begin{cases} 1 & \text{if } (v_i, v_j) \in E \\ 0 & \text{otherwise} \end{cases}$$

- The definition implies that entry in the i^{th} row and j^{th} column of the matrix is 1 if a link exists between the i^{th} and j^{th} vertex; otherwise, it is 0.
- The size $|V| \times |V|$ of adjacency matrix is independent of the number of vertices. The memory requirement is $\theta(|V|^2)$

13

Matrix Representation

Determining Number of Paths

▪ Using **adjacency matrix**, A , we can find paths of different lengths for graph. It can be shown that **square of the adjacency matrix gives paths of length 2**. If A is an adjacency matrix with element a_{ij} in i^{th} row and j^{th} column then the element p_{ij} of the A^2 is given by the following formula for the matrix multiplication

$$p_{ij} = \sum_{k=1}^n a_{ik} a_{kj}$$

▪ Computation of path length of 2 is based on the observation that if a path exists between the i^{th} and k^{th} vertex and there is path between k^{th} and j^{th} vertex, then there is a path of length 2 between i^{th} and j^{th} vertex. It can be seen that in order to compute a path of length 2 we need to multiply the adjacency matrix by itself. The elements of the resulting matrix would give number of path lengths.

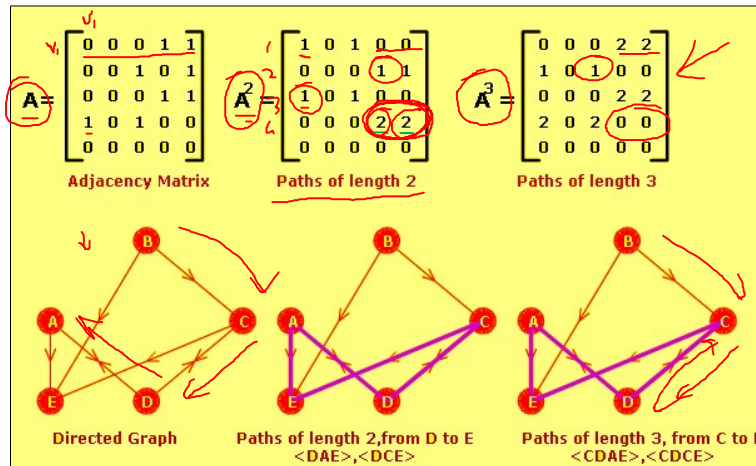
▪ In general, **r^{th} power of adjacency matrix** gives paths of **length r** between any two vertices. The computation runs in time $O(n^r)$

14

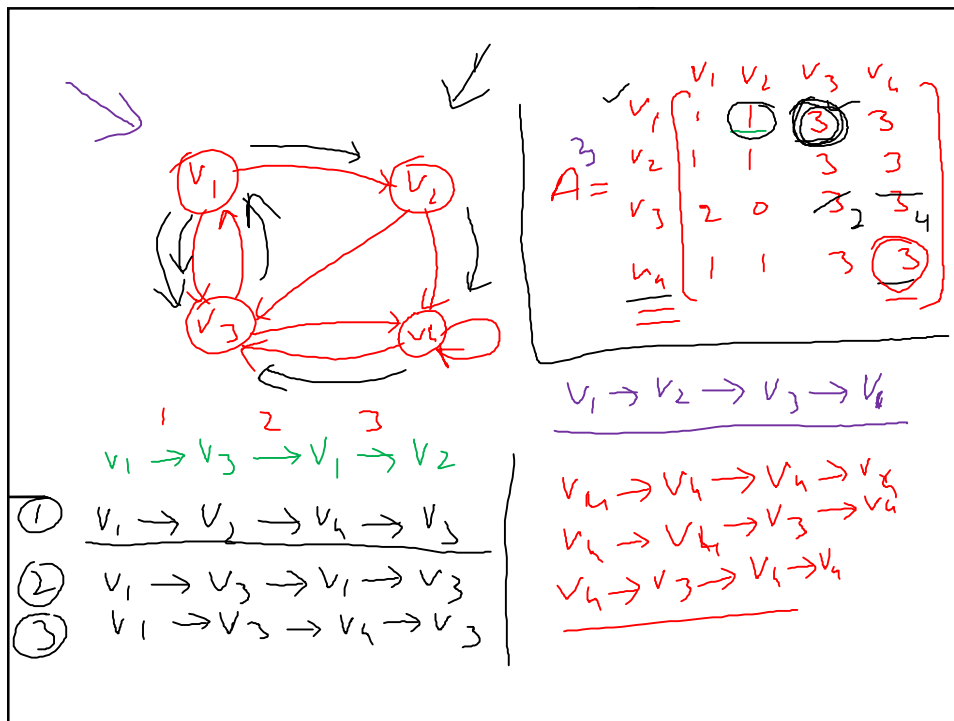
Matrix Representation

Computation of Number of Paths

- The figure illustrates the computation of **number of paths of length 2 and 3**, between **all the vertices** in a sample graph, using adjacency matrix.



15



16

Matrix Representation

Advantages and Disadvantages

The main **advantages** of matrix representation are :

- Addition of new vertices and edges is easy and efficient. It amounts to adding a new row and column and entering 1 or 0, depending whether or not a link exists.

- The **in-degree** and **out-degree** of a vertex can be computed by simply summing the corresponding column and row of the adjacency matrix. In-degree is number of edges terminating at, and out-degree the number of edges originating from, a vertex.

- The path lengths in a graph can be computed by performing matrix multiplications on the adjacency matrix.

A major **disadvantage** of matrix representation is inefficient utilization of storage space if the graph is not dense, that is, there are fewer edges. For example, the adjacency matrix of a graph with 10 vertices and four edges would contain 100 elements, only four 1's. The 96% storage space would be wasted.

17

Graphs

List Representation

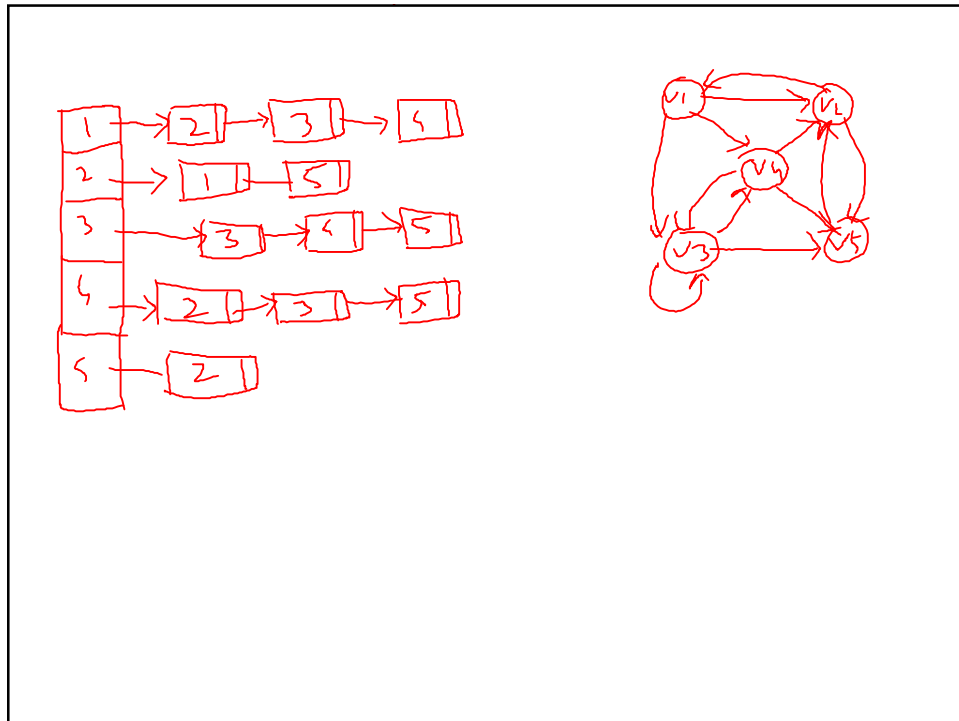
A graph can also be represented using a set of linked lists. The list representation consists of an **array of linked lists**, each corresponding to a vertex. The list stores all of the vertices that are linked to a given vertex.

- Let $G=(V,E)$ be a graph, and $Adj(u)$ be the linked list corresponding to vertex u , then for all v ,

$$v \in Adj(u), \quad \text{if } (u, v) \in E.$$

- The vertices belonging to $Adj(u)$ are called u 's immediate neighbors, or **adjacent vertices**.

18



19

List Representation

Advantages and Disadvantages

The main **advantages** of list representation are:

- The linked lists provide an efficient means for updating graphs, which require frequent insertions and deletions.
- The list can be expanded or shrunk according to increase or decrease in number of vertices and edges, and, thus, provide efficient utilization of memory for storage of a graph..

The main **disadvantage** associated with linked list representation is that search operation for a given edge is time consuming. For example, in order to search for an edge (u, v) , the set of head pointers would need to be scanned. Thereafter, the list corresponding to u would require further examination to find if it contains the vertex v .

20

Breadth First Search

Strategy

The **Breadth First Search (BFS)** is one of the standard graph traversal methods. Each node is visited once only. The method proceeds by visiting all immediate neighbors (vertices one edge away), then visiting neighbor's neighbors (vertices two edges away), and so forth, until all reachable vertices have been visited.

▪ If a graph has any **subgraph** (which is not reachable from the initial vertex), the procedure is re-started by choosing a vertex from the unvisited sub-graph.

▪ A **queue** is used to implement the BFS procedure. It keeps track of vertices to be visited next. A vertex pushed to the queue is said to be in **ready** state. An **enqueued** vertex is said to be in **wait** state. The **dequeued** vertex is referred to as **processed**.

▪ The queue is initialized with the address of starting vertex.

21

Breadth First Search

Algorithm

The **Breadth First Search (BFS)** for a connected graph consists of following steps:

Step #1: Initialize all vertices to mark as unvisited

Step #2: Select an arbitrary vertex, add it to queue

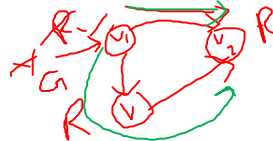
Step #3: Repeat steps # 4 through Step #5 until queue is empty

Step #4: Remove an element from the queue. Process and mark it as visited

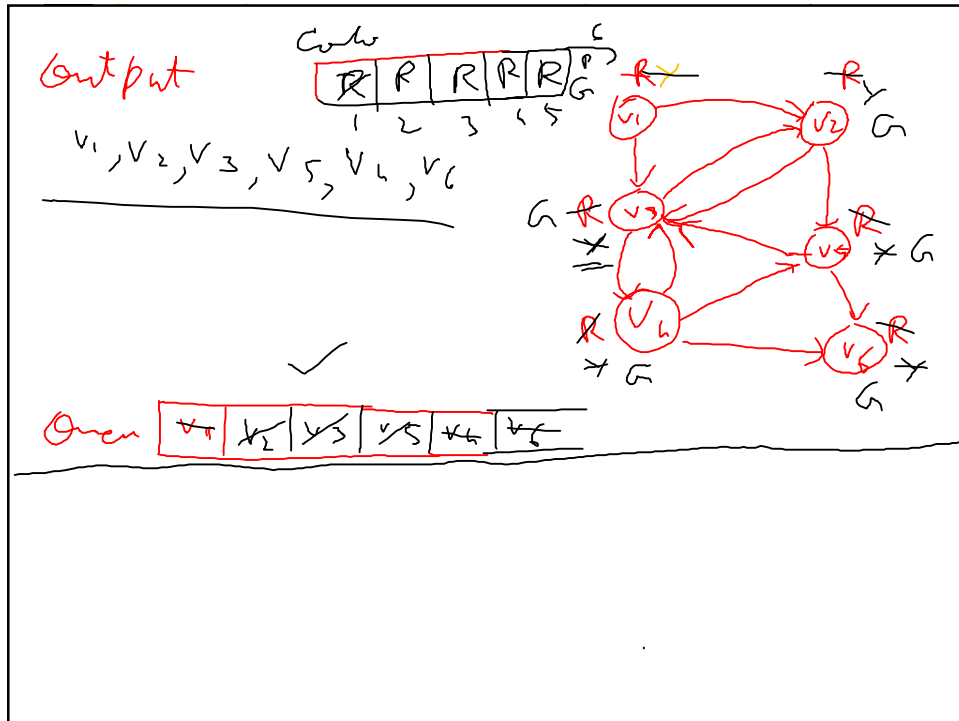
Step #5: Enqueue all adjacent vertices of the processed vertex. Go to Step # 3.

▪ The same algorithm is applied to a subgraph, which contains any unvisited vertices.

V_1



22



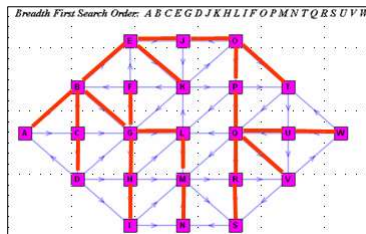
23

Breadth First Search

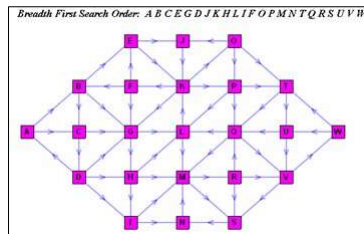
Applications

The BFS has the following useful properties:

- The edges between consecutively visited vertices form a **free tree** called **BFS spanning tree**, as shown in the figure



BFS Spanning Tree



Breadth First Search

- The BFS can find the number of disconnected components of a graph

24

Breadth First Search

Pseudo Code

The BFS implements **breadth-first-search procedure** by using a queue Q . The address s of first selected vertex is passed to the method. The implementation uses three color codes to keep track of the state of vertices: GREEN for **unvisited vertex**, YELLOW for vertex in **wait state**, PINK for **processed vertex**. The graph is represented by **adjacency list Adj**

```

BFS(G, s)                                ▶ s is address of starting vertex
for each vertex  $u \in V[G] - \{s\}$  ▶  $V[G]$  is vertex set of graph G.  $V[G] - \{s\}$  is the
                                         difference set that excludes element  $s$  in  $\{s\}$ .
2   do color[u] ← GREEN                    ▶ All unvisited vertices are assigned color GREEN
3   color[s] ← YELLOW ▶ Vertex s is being pushed to queue and assigned color YELLOW
4    $Q \leftarrow \emptyset$                     ▶ Queue is initialized as empty
5   ENQUEUE(Q,s)                            ▶ The starting vertex s is added to the queue
6   while  $Q \neq \emptyset$  ▶ A loop is created to remove vertices from the queue.
7   do  $u \leftarrow \text{DEQUEUE}(Q)$  ▶ u is the removed vertex
8       for each  $v \in \text{Adj}[u]$  ▶ Examine all vertices v that are adjacent to vertex u
9       do if color[v] = GREEN ▶ If the neighbor vertex v is not visited,
10          then color[v] ← YELLOW ▶ then assign it color YELLOW (wait state)
11          ENQUEUE(Q,v) ▶ and add to the queue
12  color[u] ← PINK ▶ The vertex u removed from the queue is marked visited
                           and assigned color PINK
    
```

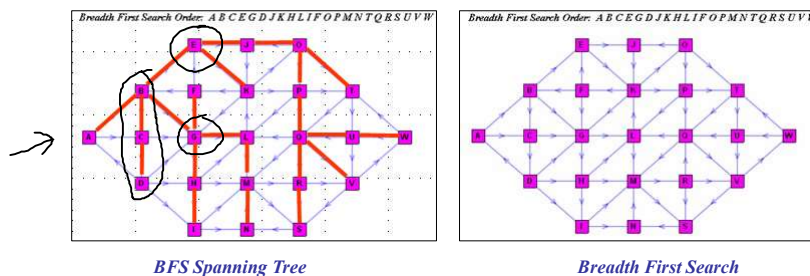
25

Breadth First Search

Applications

The BFS has the following useful properties:

- The edges between consecutively visited vertices form a **free tree** called **BFS spanning tree**, as shown in the figure



- The DFS can find the number of disconnected components of a graph

26

Breadth First Search Analysis

The running time for BFS consists of following major components

$$\begin{cases} T_i = \text{Initializing vertices } T & \leftarrow \\ T_q = \text{Queuing and Dequeuing} & \leftarrow \\ T_s = \text{Scanning Adjacency list} & \leftarrow \end{cases}$$

- The Graph $G(V,E)$ has $|V|$ vertices and $|E|$ edges. Each vertex is initialized once. Therefore, $T_i = |V|$

- The *queuing* and *dequeuing* operations take $O(1)$ time, and each vertex is queued and dequeued once,

$$T_q = 2|V|.O(1)$$

- The scanning of Adjacency list takes place for each vertex. In *worst case* all links are examined. The total time is proportional to the number of edges scanned. Since $|E|$ is the number of edges in G ,

$$T_s = |E|$$

- Thus time running time T_{BST} is given by

$$T_{BST} = |V| + 2|V|.O(1) + |E| = O(|V| + |E|)$$

27

Depth First Search

Strategy

The **Depth First Search (DFS)** is one of the standard graph traversal methods. Each node is visited once only. The method proceeds by *first visiting all distant neighbors*

- If a graph has any sub-graph (which is not reachable from the initial vertex), the procedure is re-started by choosing a vertex from the unvisited sub-graph.
- A *stack* is used to implement the DFS procedure. It keeps track of vertices to be visited next. A vertex to be pushed to the stack is said to be in *ready* state. A pushed vertex is said to be in *wait* state. The popped vertex is referred to as *processed*.
- The stack is initialized with the address of starting vertex.

28

Depth First Search

Algorithm

The **Depth First Search (DFS)** for a connected graph consists of following steps:

Step #1: Initialize all vertices to mark as unvisited

Step #2: Select an arbitrary vertex, push it to stack

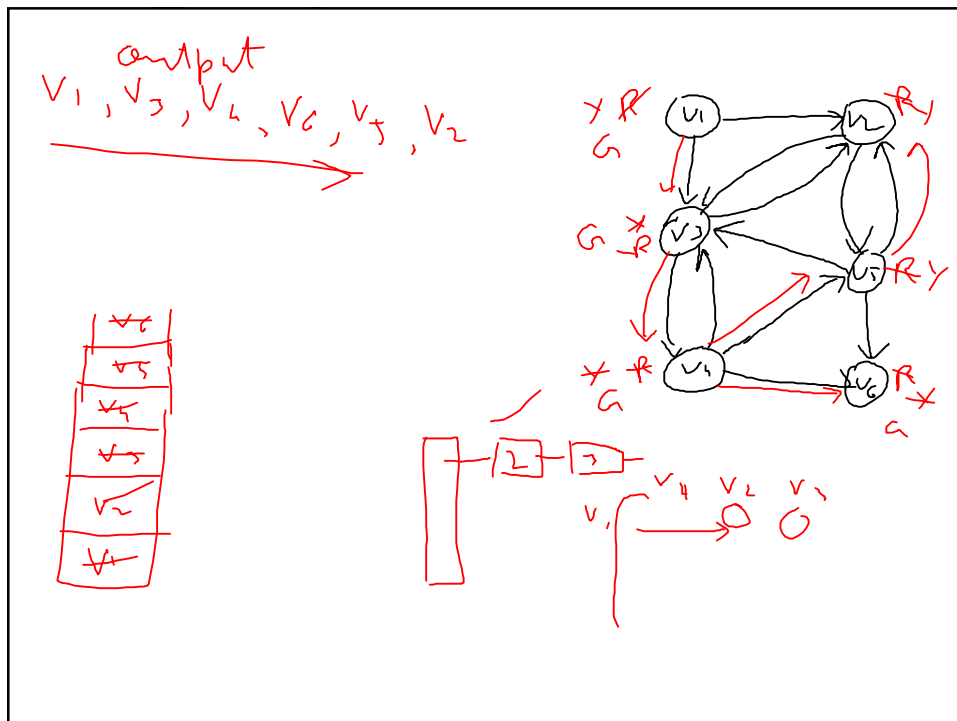
Step #3: Repeat steps # 4 through Step #5 until stack is empty

Step #4: Pop off an element from the stack. Process and mark it as visited

Step #5: Push to stack adjacent vertices of the processed vertex. Go to Step # 3.

- The same algorithm is applied to a subgraph, which contains any unvisited vertices.

29



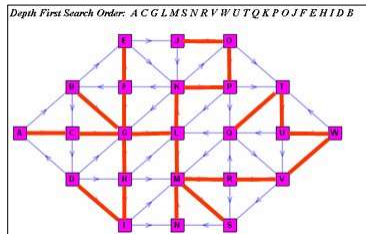
30

Depth First Search

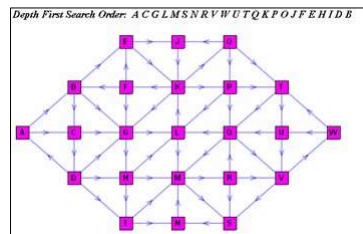
Applications

The DFS has the following useful properties:

- The edges between consecutively visited vertices form a *free tree* called **DFS spanning tree**, as shown in the figure



DFS Spanning Tree



Depth First Search

- The DFS can find *shortest distance* from the source vertex to a distant vertex
- The DFS can be used to implement the *topological sort* algorithm for a directed graph
- It can be used to find if a graph has *cycles*

31

Depth First Search

Pseudo Code

The DFS implements depth-first-search procedure by using a *stack S*. The address *s* of first selected vertex is passed to the method. The implementation uses three color codes to keep track of the state of vertices: GREEN for *unvisited* vertex, YELLOW for vertex in *wait* state, PINK for *processed* vertex. The graph is represented by *adjacency list Adj*

```

DFS(G, s)                                ▶ s is address of starting vertex
1 for each vertex u ∈ V[G] - {s} ▶ V[G] is vertex set of graph G. V[G]-{s} is the
   do color[u] ← GREEN                    ▶ V[G]-{s} is the difference set that excludes s which belongs to set {s}.
3 color[s] ← YELLOW ▶ Vertex s is being pushed to stack and assigned color YELLOW
4 P ← ∅ ▶ Stack is initialized as empty
5 PUSH(S, s) ▶ The starting vertex s is added to the queue
6 while S ≠ ∅ ▶ A loop is created to pop off vertices.
7   do u ← POP(S) ▶ u is the popped vertex
8     for each v ∈ Adj[u] ▶ Examine all vertices v that are adjacent to vertex u
9       do if color[v] = GREEN ▶ If the neighbor vertex v is not visited,
10          then color[v] ← YELLOW then assign it color YELLOW( wait state)
11          PUSH(S, v) and push it on to the stack
12 color[u] ← PINK ▶ The vertex u popped off the stack is marked visited
                    and assigned color PINK
    
```

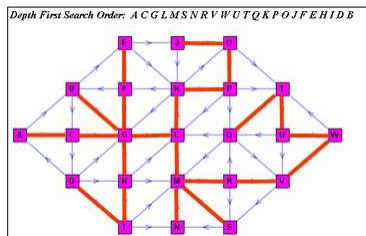
32

Depth First Search

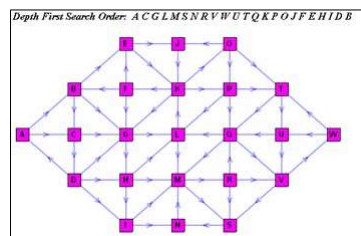
Applications

The DFS has the following useful properties:

▪ The edges between consecutively visited vertices form a **free tree** called **DFS spanning tree**, as shown in the figure



DFS Spanning Tree



Depth First Search

▪ The DFS can find **shortest distance** from the source vertex to a distant vertex

▪ The DFS can be used to implement the **topological sort** algorithm for a directed graph

33

Depth First Search Analysis

The running time for DFS consists of following major components

T_i = Initializing vertices T
 T_p = pushing and popping
 T_s = Scanning Adjacency list

▪ The Graph $G(V,E)$ has $|V|$ vertices and $|E|$ edges. Each vertex is initialized once. Therefore, $T_i = |V|$

▪ The pushing and popping operations take $O(1)$ time, and each vertex is pushed and popped once,

$$T_p = 2|V|.O(1)$$

▪ The scanning of Adjacency list takes place for each vertex. The total time is proportional to the number of edges scanned. Since $|E|$ is the number of edges in G ,

$$T_s = |E|$$

▪ Thus total running time is given by time

$$T_{DFS} = |V| + 2|V|.O(1) + |E| = O(|V| + |E|)$$

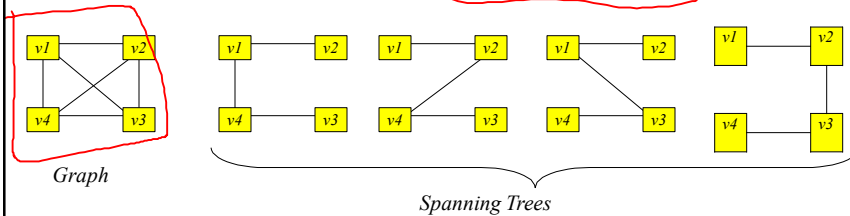
34

Graphs

Spanning Trees

A **spanning tree** for an undirected graph is a sub-graph which includes all vertices but has no cycles.

- There can be **several spanning trees** for a graph. Figure shows some of the trees for the graph with vertices $v1, v2, v3, v4$



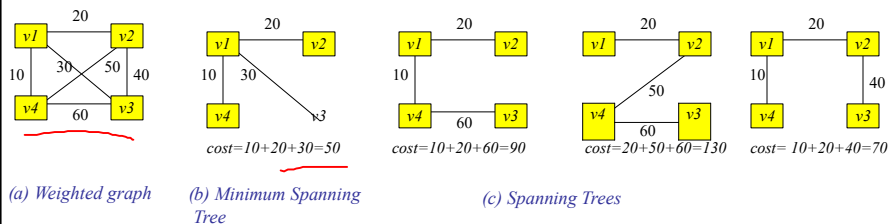
- Each tree has same number of edges
- Spanning trees can be generated by depth-first-search and breadth-first-search procedures.

35

Graphs

Minimum Spanning Trees

- A weighted undirected graph can have several spanning trees.
- One of the spanning trees has smallest sum of all the weights associated with the edges. This tree is called minimum spanning tree (mst).
- Figure shows a sample weighted graph, some of the **spanning trees**, and the **minimum spanning tree**.



36

Graphs

Minimum Spanning Trees

Minimum spanning trees have many practical applications. Some typical applications are:

- A telephone network can be configured, using minimum spanning tree, to have minimum cable length.
- The air travel routes can be selected so that the travel time or travel cost is least.
- A computer network can be set up with minimum routing distance
- Linking a group of island with bridges so that total bridge span length is minimum

Two important algorithms for creating a minimum spanning tree for a graph, named after their inventors, are Kruskal's algorithm and Prim's algorithm.

37

Minimum Spanning Tree

Kruskal's Algorithm

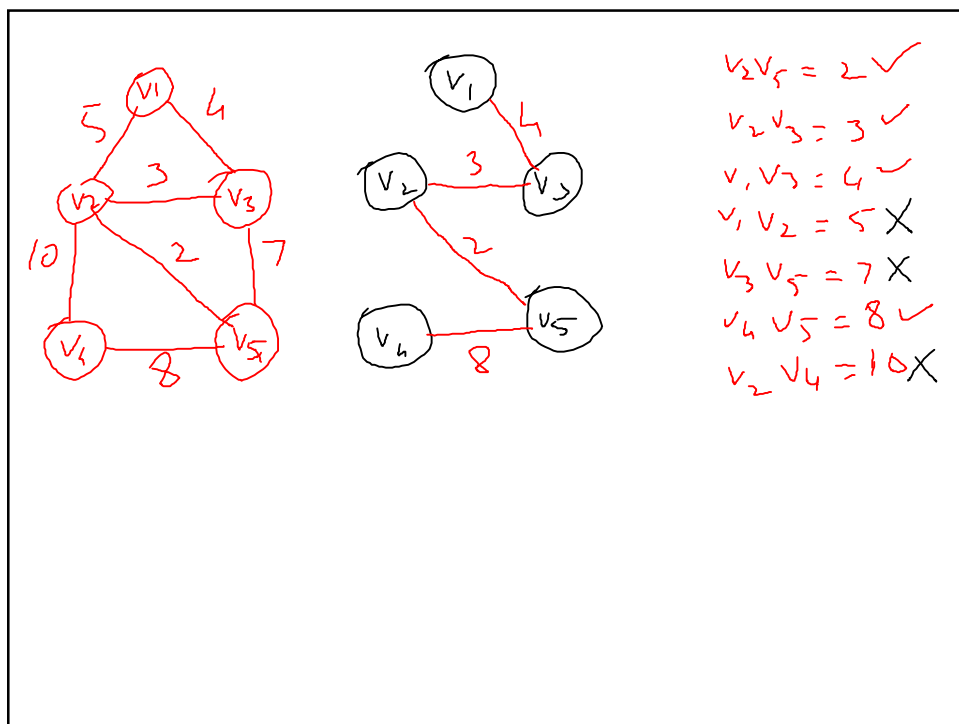
The **Kruskal's algorithm** works as follows:

- Step #: 1 Remove all edges of the graph
- Step #: 2 Arrange edges according to their weights
- Step #: 3: Select a edge with least weight
- Step #: 4: Attach the edge to the corresponding vertices if it does not form cycle; otherwise, drop the edge
- Step #: 5: Repeat steps 3 to 4 until all the edges are processed (added or dropped)

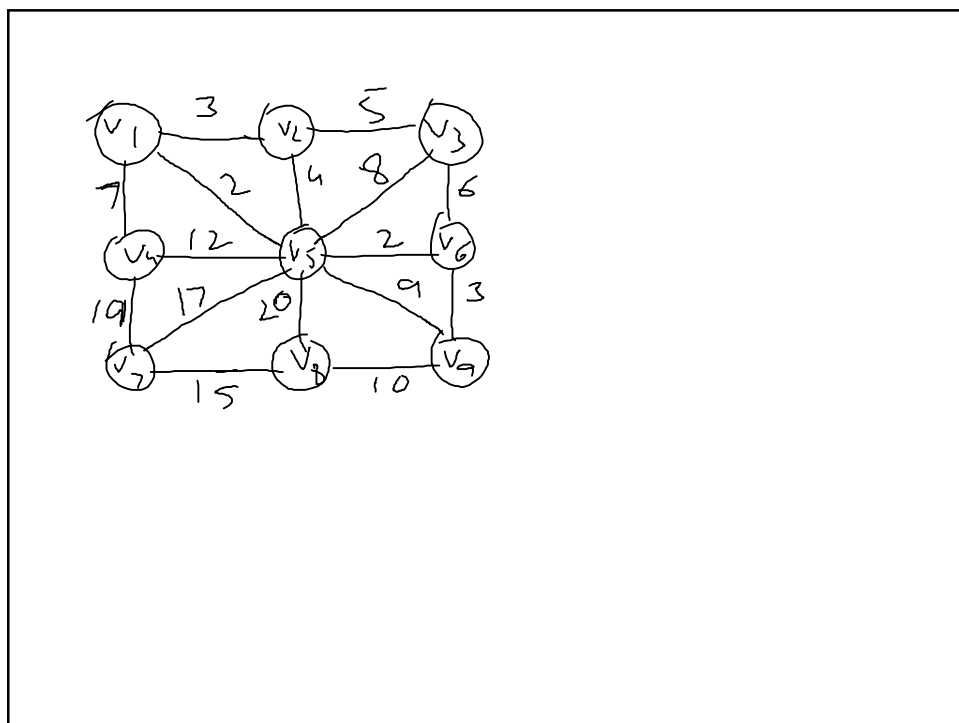
•Kruskal's algorithm is categorized as **greedy**, because at each step it picks an edge with least weight.

•The Kruskal's algorithm can be implemented in several ways. Generally, a **priority queue** is used to store graph edges, so that edges are retrieved in increasing order.

38



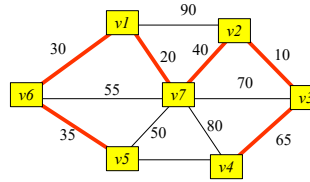
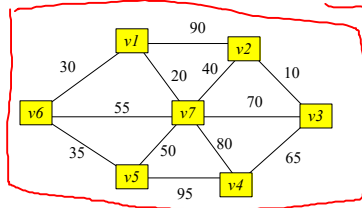
39



40

Kruskal's Algorithm

Using Set Union



Edge	Weight	Edge	Action	UV={v1, v2, v3, v4, v5, v6, v7}
(v2,v3)	10	(v2,v3)	added	{v1, v2, v3, v4, v5, v6, v7}
(v1,v7)	20	(v1,v7)	added	{v1, v7, v2, v3, v4, v5, v6}
(v1,v6)	30	(v1,v6)	added	{v1, v6, v7, v2, v3, v4, v5}
(v5,v6)	35	(v5,v6)	added	{v1, v5, v6, v7, v2, v3, v4}
(v2,v7)	40	(v2,v7)	added	{v1, v2, v3, v5, v6, v7, v4}
(v5,v7)	50	(v5,v7)	rejected	
(v6,v7)	55	(v6,v7)	rejected	
(v3,v4)	65	(v3,v4)	added	{v1, v2, v3, v4, v5, v6, v7}
(v3,v7)	70	(v3,v7)	rejected	
(v4,v7)	80	(v4,v7)	rejected	
(v1,v2)	90	(v1,v2)	rejected	
(v4,v5)	95	(v4,v5)	rejected	

41

Kruskal's Algorithm

Pseudo Code

Following is pseudo code for *Kruskal's algorithm*.

MST-KRUSKAL(G,W)

```

1   $T \leftarrow \emptyset$  ▶ Initialize Minimum Spanning Tree
2   $VS \leftarrow \emptyset$  ▶ Initialize Union set VS
3  for each vertex  $v \in V$ 
4       $do VS \leftarrow VS \cup \{v\}$  ▶ Add Vertex sets to union set
5  for each  $(u, v) \in E$  ▶ Create a priority queue of edges and weights
6       $do ENQUEUE(Q, u, v, w)$  ▶ weights are keys (priorities)
7  while  $|VS| > 1$  do ▶ Continue until all vertices form a single set of cardinality 1
8       $(u, v) \leftarrow DEQUEUE(Q)$  ▶ Remove an edge (u,v) from the queue
9      if  $u, v$  belong to different sets  $VS1 \in VS$  and  $VS2 \in VS$ 
10         then  $VS \leftarrow VS - VS1$  ▶ remove set VS1 from VS
11              $VS \leftarrow VS - VS2$  ▶ remove set VS2 from VS
12              $VS \leftarrow VS \cup (VS1 \cup VS2)$  ▶ Join sets VS1 and VS2, add the union to VS
13              $T \leftarrow (u, v)$  ▶ Add the selected edge to the spanning tree
14 return  $T$ 
    
```

42

Kruskal's Algorithm Analysis

Assuming that **priority queue** is implemented as a heap. The time to sort edges is $O(|E| \cdot \lg |E|)$.

▪ The initialization of VS takes $O(|V|)$.

▪ The total time to find sets VS1 and VS2 and replacing them by their union is at most $O(|E|)$

Thus, on the whole Kruskal's algorithm runs in time $O(|E| \cdot \lg |E|) + O(|E|) + O(|V|)$

$$O(|E| \lg |E| + |E| + |V|)$$

43

Minimum Spanning Trees

Prim's Algorithm

The **Prim's algorithm** is an alternative method for creating minimum spanning trees for a **weighted undirected graph**. Unlike the Kruskal's algorithm, the Prim's algorithm makes a systematic selection of vertices. It proceeds by choosing a vertex, which is at shortest distance of all the linked vertices.

➤ Let V be the vertex set for a graph G . Let T be the minimum spanning tree for G . The Prim's algorithm proceeds as follows:

Step #1: Select some vertex s in V , as the starting vertex.

Step #2: Add vertex s to an empty set S . Remove s from V .

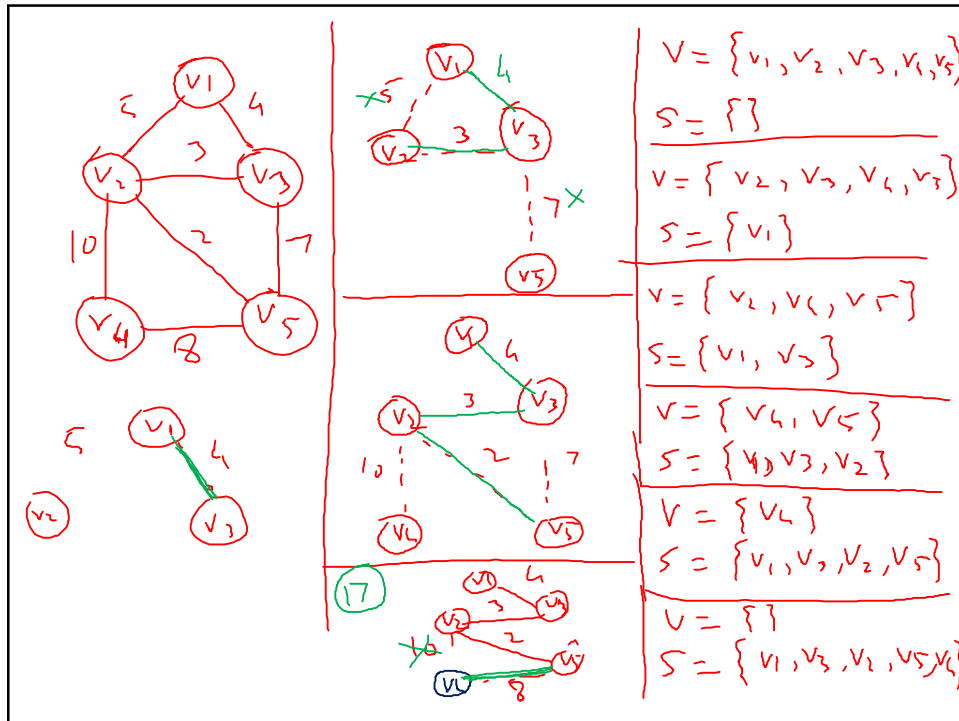
Step #3: Repeat Step #4 through Step #6 until the set V is empty.

Step #4: Examine all vertices in S which are linked to vertices in V .

Step #5: Choose the vertex u in V which has the minimum distance from vertex v in S .

Step #6: Remove vertex u from V and add it to S . Move edge (v, u) to T .

44



45

Prim's Algorithm

Pseudo Code

The MST-PRIM method implements Prim's algorithm for creating **minimum spanning tree** for a graph G . An initial starting vertex s is passed to the method.

MST-PRIM(G, s)

```

1  $T \leftarrow \emptyset$   $\triangleright T$  is set of vertices for minimum spanning tree
2  $S \leftarrow \emptyset$   $\triangleright S$  contains selected vertices
3  $S \leftarrow S \cup \{s\}$   $\triangleright$  starting vertex is added to  $S$ 
4  $V \leftarrow V - \{s\}$   $\triangleright$  Starting vertex is removed from  $V$ 
5 while  $V \neq \emptyset$  do
6    $d \leftarrow \infty$   $\triangleright d$  is used to store minimum distance. It is initialized to some very large number
7   for each vertex  $u \in S$  do  $\triangleright$  select a vertex  $u$  in  $S$ 
8     for each vertex  $v \in \text{Adj}[u]$  do  $\triangleright$  Examine all vertices which are linked to  $u$ 
9       if  $d > w(u,v)$   $\triangleright$  Check if the vertex  $u$  has shorter distance
10        then  $d \leftarrow w(u,v)$   $\triangleright$  If yes, replace  $d$  with weight  $w(u,v)$ 
11         $w1 \leftarrow v$   $\triangleright$  store vertex  $v$  in  $w1$ 
12         $w2 \leftarrow u$   $\triangleright$  store vertex  $u$  in  $w2$ 
13    $V \leftarrow V - \{w1\}$   $\triangleright$  remove vertex  $w1$  from  $V$ 
14    $S \leftarrow S \cup \{w1\}$   $\triangleright$  add vertex  $w1$  to  $S$ 
15    $T \leftarrow T \cup \{(w1, w2)\}$   $\triangleright$  Add edge  $(w1, w2)$  to  $T$ 
16 return  $T$ 

```

46

Single Source Shortest Paths

Dijkstra's Algorithm

47

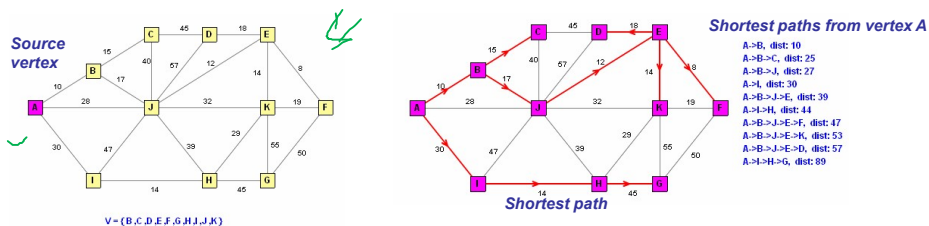
Single Source Shortest Paths

Dijkstra's Algorithm

The algorithm for single-source shortest paths determines the smallest routes of all vertices in a graph from a given vertex, called source vertex.

- Figure shows a sample graph and the shortest paths from a vertex A.

Weighted Directed Graph



- The shortest path algorithm has several applications. It can be used, for example, to find shortest route between two cities

- An efficient algorithm for single-source shortest path is known as *Dijkstra's algorithm*, named after its inventor.

48

Graph

Dijkstra's Algorithm

Dijkstra's algorithm determines shortest paths from a single vertex in a **weighted directed graph**. The working of Dijkstra's algorithm is similar to Prim's algorithm. It repeatedly computes the distances between a source vertex and the vertices being explored, and selects the minimum distance from among the feasible paths. It is categorized as greedy algorithm.

Let V be the vertex set for a graph G . Let d be an array that stores shortest distances. The algorithm proceeds as follows:

Step #:1 Initialize elements to some large number

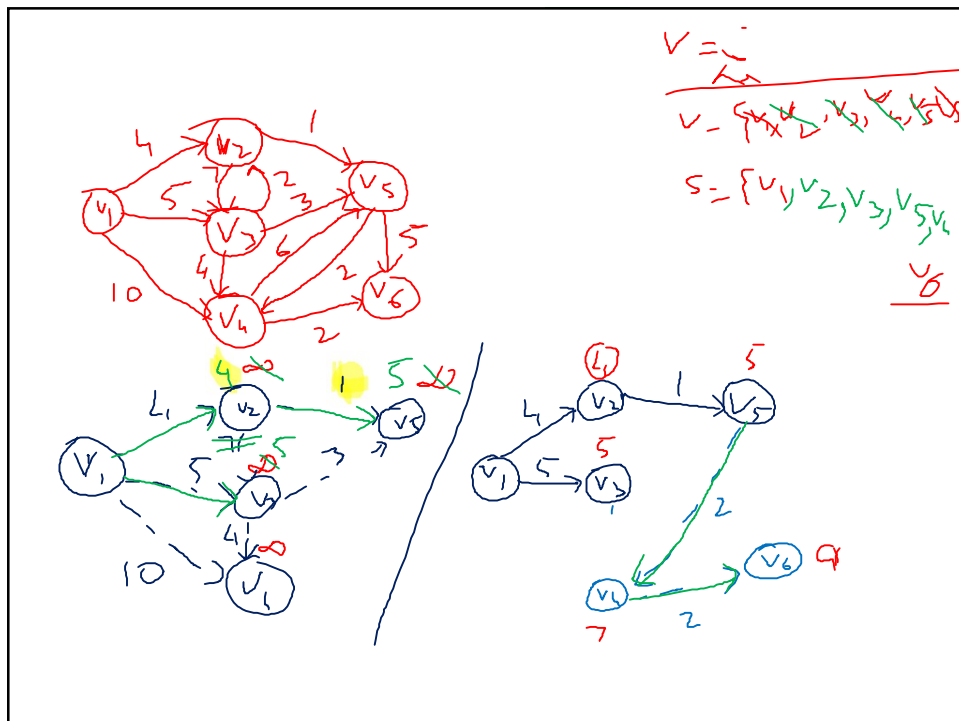
Step #:2 Add source vertex s to an empty set S . Remove s from V .

Step #:3 Repeat Step #:4 through Step #:5 until the set V is empty.

Step #:4: Examine vertices in S which are linked to vertices in V . Choose the vertex u in V which has the shortest distance from the source. Store this distance in $d[u]$

Step #:5: Remove vertex u from V and add it to S .

49



50

Dijkstra's Algorithm

Pseudo Code

The pseudo code for Dijkstra's algorithm is follows:

DIJKSTRA(G, s)

```
1 for each vertex  $v \in V$ 
2 do  $d[v] \leftarrow \infty$  ▶ Array  $d[V]$  stores minimum distances .
3  $S \leftarrow \emptyset$  ▶  $S$  contains selected vertices
4  $S \leftarrow S \cup \{s\}$  ▶ starting vertex is added to  $S$ 
5  $V \leftarrow V - \{s\}$  ▶ Starting vertex is removed from  $V$ 
6 while  $V \neq \emptyset$  do
7   for each vertex  $u \in S$  do ▶ select a vertex  $u$  in  $S$ 
8     for each vertex  $v \in \text{Adj}[u]$  do ▶ Examine all vertices which are linked to  $u$ 
9       if  $d[u] > d[u] + w(u,v)$  ▶ Check if the  $d[u]$  is smaller
10        then  $d[u] \leftarrow d[u] + w(u,v)$  ▶ If yes, replace  $d[u]$  with sum of  $w(u,v)$ 
11         $w \leftarrow v$  ▶ store vertex  $v$  in  $w$ 
12  $V \leftarrow V - \{w\}$  ▶ remove vertex  $w$  from  $V$ 
13  $S \leftarrow S \cup \{w\}$  ▶ add vertex  $w$  to  $S$ 
```